File handling/Operation:

File handling in Python allows you to work with files on your computer's file system. You can

1) Create the File
2) Open file
3) read from files
4) write to files
5) Delete files

and perform various other file-related operations. Python provides built-in functions and methods for file handling. Here are the different methods commonly used for file handling in Python:

Writing to a file:

```
filePath = "D:/neData/abc.txt"
file  = open(filePath,"w")
```

Some important Modes of File Operations:

'r' : Open for reading (default).
'w' : Open for writing.
'a' : Open for writing, appending the data in the file.

Writing to a file:

- **write()** : Writes a string to the file.
- **writelines()** : Writes a sequence of strings to the file.

```
file.write("this is text file")
```

Reading from a file:

- **read()** : Reads the entire contents of the file as a single string.
- **readline()** : Reads a single line from the file.
- **readlines()** : Reads all lines from the file and returns them as a list.

```
file  = open(filePath,"r")
abc = file.read()
abc = file.readline()
abc = file.readlines()
```

Reading from a file:

Use write function to do the append in file

```
file.write("-----------------------------------------------------------------\n")
file.write("Hello, this is some text written to a text file using Python!\n")
file.close()
```

Closing a file:

- **close()** : Closes the file. It's important to close files when you're done working with them to release system resources.

Removing a file:

- **remove()** : Removes (deletes) the file from the file system.

File Objects and Modes of File Operations:

File Objects: In Python, files are represented by file objects. You can create a file object using the **open()** function.

Modes of File Operations:

'r' : Open for reading (default).

'w' : Open for writing.
'a' : Open for writing, appending to the end of the file if it exists.
'b' : Binary mode.
't' : Text mode (default).
'+' : Open for updating (reading and writing).

Use of **with** Keyword:
- Simplification:
- Release of resources:

In Python, the **with** statement is used in file handling to ensure that resources are properly managed and automatically released when they are no longer needed.

It **simplifies** the syntax for opening and working with files by taking care of closing the file for you, even if an exception occurs the **with** keyword,

```
with open(filePath,'w') as file:
    file.write("asdfasdfasdf")
    file.write("asdfasdfasdf")
    file.write("asdfasdfasdf")
    file.write("asdfasdfasdf")
```

Examples of files can be handled with Python.

Text Files:
- **.txt** : Plain text files.
- **.csv** : Comma-separated values files.
- **.json** : JSON files.

Binary Files:
- **.jpg, .jpeg** : JPEG image files.
- **.zip, .tar, .gz** : Compressed files.
- **.docx, .xlsx, .pptx** : Microsoft Office files.
- **.sqlite** : SQLite database files.

Structured Data Files:
- **.xls, .xlsx** : Excel files.
- **.dbf** : dBASE database files.
- **.sav** : SPSS data files.

Configuration Files:
- **.cfg, .conf** : Configuration files.

Code Files:
- **.py** : Python source code files.
- **.html, .css, .js** : Web development files.

Data Exchange Files:
- **.yaml, .yml** : YAML files.
- **.toml** : TOML files.

Game Files:
- **.dat** : Game data files.
- **.sav** : Saved game files.